Ders 5 Genişletilmiş Notu: Algoritma İyileştirme: Gauss, Hızlı Üs, Pi ve e^x Serileri, Karşıt Örnek, Derleme Aşamaları

Ders 5 Genişletilmiş Notu: Algoritma İyileştirme: Gauss, Hızlı Üs, Pi ve e^x Serileri, Karşıt Örnek, Derleme Aşamaları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Planı ve 1'den N'e Kadar Toplam
- 2. Gauss Formülü: Sabit İşlem Sayısı
- 3. Yöntemi Başka Probleme Uyarlamak: Faktöriyel
- 4. Stirling Formülü: Yaklaşık Çözümün Bedeli
- 5. Hızlı Üs Alma
- 6. Akış Diyagramı, Sözde Kod ve Program Kodu
- 7. Pi Sayısını Seriyle Hesaplamak
- 8. Pi Algoritmasını Adım Adım İyileştirmek
- 9. Pi Programının C ile Yazılması
- 10. İki Sürüm Arasındaki Trade-off
- 11. e^x Serisi
- 12. Fibonacci Sayıları
- 13. Basamaklarının Küpleri Toplamı Kendine Eşit Olan Sayılar
- 14. Karşıt Örnekle Çürütme
- 15. $a^4 + b^4 + c^4 = d^4$ Önermesi
- 16. Her Çift Sayı İki Asalın Toplamı mıdır?
- 17. Bir C Programının Derlenme Aşamaları
- 18. `scanf` ve Giriş Tamponu

1. Dersin Planı ve 1'den N'e Kadar Toplam

Bu derste Fatih Hoca'nın ders notlarındaki algoritma soruları çözülüyor ve program kodu yazılmaya başlanıyor. Her soruda aynı yol izleniyor: önce akla gelen en basit çözüm kuruluyor, sonra bu çözümün kaç işlem ve ne kadar bellek harcadığı hesaplanıyor, en sonra gereksiz işler ayıklanıyor.

İlk soru, daha önce konuşulan Gauss formülüyle ilgili. Hoca, formülün ardışık sayıları toplamayı sağladığını hatırlattı: $\frac{n(n+1)}{2}$. Sonra şunu sordu: Hiç Gauss bilmiyor olsaydık 1'den n'e kadar sayıların toplamını nasıl bulurduk?

1.1 Döngülü Algoritma

Akış diyagramının adımları şöyle:

1. Başla.
2. n sayısını kullanıcıdan al.

3. `toplam` ve `i` değişkenlerini oluştur, değerlerini sıfır yap.
4. `i`'yi 1'den başlat. `i <= n` olduğu sürece `toplam = toplam + i` yap, `i`'yi bir artır.
5. `toplam`'ı ekrana yaz.
6. Dur.

Kod olarak düşünürsek döngü `for (i = 1; i <= n; i++) toplam = toplam + i;` biçimindedir. Hoca niyetin zaten bu olduğunu söyledi: Yapmak istediğimizi söylediğimiz anda algoritma ortaya çıkıyor. Şimdi bu algoritmanın maliyetini hesaplayıp düzeltmeye çalışacağız.

1.2 İşlemlerin Sırası

Bir `for` döngüsünde işlemler şu sırayla yapılır. Başlangıç ataması T anında, yani T saat çevriminde (clock cycle) olsun:

- T: `i = 1` ataması
- $T+1$: `i <= n` karşılaştırması
- $T+2$: `toplam + i` toplaması
- $T+3$: `toplam`'a atama
- $T+4$: `i++` artırması
- $T+5$: yeniden karşılaştırma

Sonrasında aynı dört adım tekrar eder. Hoca bu tekrarı dört adımlık bloklar hâlinde yazdı: bir turu $4j+1$, $4j+2$, $4j+3$, $4j+4$ diye numaraladı. Başta `i` dediği sayaca, döngü değişkeniyle karışmasın diye `j` dedi. Başlangıç ataması ise bir kez olur ve biter. Döngü burada olsun olmasın, başlangıç kısmı iki ya da üç kez yapılmaz.

1.3 İşlem Sayısı

Tekil işlemleri sayalım:

- `i = 1` ataması: 1 kez.
- `i <= n` karşılaştırması: $n+1$ kez. 1'den n'e kadar n sayı var, ama program $n+1$. denemede şartın sağlanmadığını görüp döngüden çıkar.
- `toplam + i` toplaması: n kez, çünkü döngü n kez döner.
- `toplam`'a atama: n kez.
- `i++` artırması: n kez.

Hoca önce `i++` için de $n+1$ dedi ve toplamı $4n+3$ buldu. Bir öğrenci itiraz etti.

Soru: `i`'yi bir artırırken 1'den 2'ye birinci adım, 2'den 3'e ikinci adım diye sayarsak n tane olmuyor mu? **Cevap:** Evet. $n = 10$ için 1'den 10'a 9 artırma yapılır, 10'dan 11'e bir artırma daha yapılır. Toplam 10 artırma, yani n tane. Hoca itirazın doğru olduğunu söyledi ve sayımı düzeltti.

Soru: Karşılaştırmayı neden n değil $n+1$ aldık? **Cevap:** Döngü, artırmadan sonra şartı her defasında yeniden kontrol eder. Kaç kez döneceğini kendisi bilmez. $n+1$. değeri de test eder, şart yanlış çıkınca döngüden çıkar.

Düzeltilmiş toplam:

$$1 + (n+1) + n + n + n = 4n + 2$$

$n = 10$ için $4 \cdot 10 + 2 = 42$ işlem, $n = 1000$ için $4 \cdot 1000 + 2 = 4002$ işlem yapılır. Sınıfta 40.002 cevabı da söylendi, hoca bunun olmayacağını belirtti. İşlem sayısı n ile doğrusal (lineer) büyür.

Çalışma zamanına $T_1(n)$ dersek, adım sayısı için $S(n)$ de denebilir, grafiği $n = 0$'da 2'den başlayan, eğimi 4 olan bir doğrudur.

Hoca bu sayımda bir kabul yaptığını da belirtti: Atama, toplama ve ++ gerçekte farklı sürelerde çalışır. Biz bunların maliyetini eşit kabul ediyoruz, bu ayrıma ciddi şekilde girmeye gerek yok.

2. Gauss Formülü: Sabit İşlem Sayısı

Gauss'a göre `toplam = n * (n + 1) / 2` yazmak yeterlidir. Aynı kabulle sayarsak bir çarpma, bir toplama, bir bölme, bir atama, yani toplam 4 işlem vardır. Buna $T_2(n) = 4$ diyelim. $n = 10$ de alsak, 1000 de alsak, 1 milyar da alsak işlem sayısı 4'tür. Grafiği yatay bir doğrudur.

n'in sonsuza giden değerlerini düşünelim. Hoca buradaki sonsuzu "belleğinizi dolduracak kadar sonsuz" diye tarif etti. İkinci algoritma sabit kalırken birinci algoritma çok uzun zaman gerektiren bir işleme doğru yelken açar. Demek ki Gauss'u kullanmak daha iyidir.

3. Yöntemi Başka Probleme Uyarlamak: Faktöriyel

Hocaya göre problem çözmede ve bilimde işler şöyle ilerler: Birisi bir şey yapar. Ondan yola çıkarak ya açığını yakalarsınız ya da yöntemi başka bir yere uyarlamaya çalışırsınız. Fatih Hoca'nın notlarındaki soru da budur: Gauss'u biraz zorlarsak başka problemlerde işimize yarar mı?

Az önce yaptığımız şey aslında $\sum_{i=1}^{n} i$ sembolünün yaptığı iştir. Aynı benzetmeyle $\prod_{i=1}^{n} i$ sorulursa $1 \cdot 2 \cdots n = n!$ bulunur. Algoritmanın farkı çok azdır: Sonuç değişkeni 1'den başlar, toplama yerine çarpma kullanılır. Dolayısıyla kod yazmadan söyleyebiliriz ki işlem sayısı aynı kalır: $T_{1c}(n) = 4n + 2$.

4. Stirling Formülü: Yaklaşık Çözümün Bedeli

Faktöriyelin Gauss formülü gibi kesin bir kısa yolu yok, ama başka bir çözümü var. Hoca bu çözümün algoritmanın bir tarafına fayda sağlarken öbür tarafına sağlamadığını, belki zarar bile verdiğini söyledi. Bu çözüm Stirling formülüdür:

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$

4.1 İşlem Sayısı

Formüldeki işlemleri hoca şöyle saydı:

- $2 \cdot \pi$ bir çarpma, sonucun n ile çarpılması ikinci çarpma.
- Karekök üçüncü işlem.
- Karekök sonucunun $\left(\frac{n}{e}\right)^n$ ile çarpılması dördüncü işlem.
- $\frac{n}{e}$ bölmesi bir birim. n ve e belli olduğu için bu bölme bir c sayısı verir.
- $c \cdot c \cdot c \cdots$ diye n tane çarpma.

Hoca sonucu $T_2(n) = n + 4$ olarak verdi. Yani $4n + 2$'den $n + 4$'e inildi.

> **Ders sonrası not:** Maddelerdeki dört işleme bölme de ayrı eklenip n çarpma alınırsa toplam $n+5$ olur. c^n için $n-1$ çarpma alınırsa $n+4$ çıkar. Bu fark büyümenin doğrusal olduğu sonucunu değiştirmez.

4.2 Doğrusaldan Yavaş Büyüyen Fonksiyonlar

Hoca $4n$ ile n arasında kayda değer bir fark olmadığını vurguladı: İkisi de n artınca sonsuza gider. Bize n'in artan değerleri için sonsuza gitmeyen, mümkünse sabit kalan ya da sonsuza daha yavaş giden fonksiyonlar lazımdır. Doğrusaldan daha yavaş büyüyen fonksiyon logaritmik fonksiyondur. Sabit fonksiyon ise hiç ilerlemez. Eğer n gibi büyüyen bir algoritmayı $\log n$ gibi ya da c gibi büyüyen bir algoritmaya dönüştürebilirsek işimize yarar.

4.3 Bellek ve Doğruluk

Bellek açısından iki çözüm arasında fark yoktur: Klavyeden alınan n ve sonuç olmak üzere iki değişken yeter. İşlem sayısı da çok değişmedi. O hâlde başka ne değişti? Çözüm artık yaklaşıktır, sonuçta kesme hatası vardır. Bir yerden alırken bir yerden kaybettik.

> **Hocanın vurgusu:** Algoritmanın en önemli özelliklerinden biri doğruluktur, yani ne kadar kesin çözüm (exact solution) verdiğidir. Probleminiz kesin çözüm olmadan çalışmıyorsa bu algoritma çöp olur. Her zaman tam sonuç gerekmiyorsa kullanılabilir.

4.4 Yaklaşık Çözüm Nerede İşe Yarar: Faktöriyelin Türevi

Yaklaşık çözümün işe yaradığı yere örnek olarak hoca $n!$'in türevinin nasıl alındığını sordu. Sınıfta birkaç öneri çıktı.

Soru: $n!$'i $n \cdot (n-1) \cdots$ diye açıp belirli bir noktadaki türevi alsak olur mu? **Cevap:** Bu faktöriyelin hesaplanmasıdır, oysa türev isteniyor. x^4'te olduğu gibi sayılar kullanmadan fonksiyon olarak istersem bu olmaz. Burada ayrık (discrete) düşünülüyor, oysa sürekli düşünmek gerekiyor. Aynı öğrenci iki tarafın ln'ini almayı da önerdi, ama sonucunu bilmediğini söyledi.

Soru: Türevin olması için fonksiyonun reel sayılarda geçerli olması gerekmiyor mu? Faktöriyel yalnızca tam sayılar için geçerli. **Cevap:** Söylenen doğrudur. Durum ayrık olduğu için tam olarak türev almak mümkün değildir. Birçok yöntem olmakla beraber yaklaşık türev alınır: Stirling formülünün n'e göre türevi alınırsa bu, $n!$'in türevi olarak kullanılabilir.

Öyleyse yaklaşık durumlar bazen işimize yarıyor.

> **Ek bilgi (derste anlatılmadı):** Faktöriyelin reel sayılara genişletilmiş hâli büyük olasılıkla Gamma fonksiyonudur: $\Gamma(n+1) = n!$. Türev bu sürekli fonksiyon üzerinden alınabilir. Stirling formülü de büyük olasılıkla bu fonksiyonun büyük değerler için yaklaşık hâlini verir.

5. Hızlı Üs Alma

Stirling'i kullanmaya karar verdik, ama hoca "hâlâ şu n'lere kafayı taktık" dedi: Bunu daha indiremez miyiz? Stirling'deki $\left(\frac{n}{e}\right)^n$ bir kuvvet hesabıdır. Bu yüzden kuvvet alma işini hızlandırmak gerekir.

5.1 Düz Döngüyle Kuvvet

n^{13}'ü hesaplayacağız. Düz çözüm şudur: `R = 1` ile başla, `for (i = 1; i <= 13; i++)` döngüsünde `R = R * n` yap. Döngü 13 kez döner.

Hoca bu noktada bir soru sordu: C'de döngüler kaçtan başlar? Sınıftan "sıfırdan" cevabı geldi.

> **Hocanın vurgusu:** Bu çok yapılan bir hatadır. Döngüler nereden istersek oradan başlar: 1'den de başlatırsınız, 13'ten de başlatırsınız, geriye de gelirsiniz. Sıfırdan başlayan dizilerdir. Problem ne istiyorsa ona bakacağız.

5.2 Basamakları Birer Birer Atlamamak

Şimdi şu iki satırı yazalım, `R` başta 1 olsun:

- `R = R * n` yapınca `R`'nin içinde n olur.
- `R = R * R` yapınca `R`'nin içinde n^2 olur.

Bir kez daha `R = R * R` yaparsak `R`'de n^4 olur. Normalde iki adımda yapacağım işi bir adımda yaptım. Hocanın benzetmesiyle 13'e gitmek için basamakları birer birer atlamaya gerek yok. Bu adımları belli bir sistematiğe göre atarsak bize bir iş çıkar.

5.3 Üssün İkilik Açılımı

Sistematik şudur: Üssün ikilik (binary) açılımına bakılır. $13 = 1101_2$. `R = 1` ile başlanır.

- Bit 1 ise: `R = R * R * n`. Karesini al, sayıyla çarp.
- Bit 0 ise: `R = R * R`. Yalnızca karesini al.

Hocanın örneğinde $n = 5$, yani 5^{13} hesaplanıyor. Değerleri "(ara adım eklendi)" olarak ekledik:

Bit	İşlem	R'nin üssü	R'nin değeri
1	`R = 1 * 1 * 5`	5^1	5
1	`R = 5 * 5 * 5`	$5^{2+1} = 5^3$	125
0	`R = 125 * 125`	$5^{3\cdot 2} = 5^6$	15625
1	`R = 15625 * 15625 * 5`	$5^{6\cdot 2+1} = 5^{13}$	1220703125

Sonuç python ile doğrulandı: $5^{13} = 1220703125$.

> **Ders sonrası not:** Hoca "sağdan başlıyoruz" dedi; yaptığı adımlar ise (n, n^3, n^6, n^{13}) bitlerin soldan sağa, yani en anlamlı bitten başlanarak okunduğunu gösteriyor. Aynı kurallar sağdan sola ($1, 0, 1, 1$ sırasıyla) uygulanırsa python denemesinde $5^{11} = 48828125$ çıktı. Bu yüzden bu sürümde okuma yönü soldan sağadır.

Soru: 1101'e göre yapmamızın sebebi 13'ün ikilik açılımının bu olması mı? **Cevap:** Evet, o olmadan olmaz. İnanmayan başka sayılar için denesin.

5.4 Adım Sayısı: $\log n$

13 kez dönecek iş 4 turda bitti. Hoca bunu genelleştirdi: Ayrıntısı özyinelemeli (recursive) algoritmalarda kanıtlanmak üzere bu yöntem $\log n$ kez işlem yapar. Böylece Stirling'de $n + 4$'ten $\log n + 4$'e iniyoruz.

Karşılaştırma için hocanın verdiği iki değer:

- 1024 elemanda: 1024 adımlık iş 10 adıma iner, çünkü $\log_2 1024 = 10$.
- $n = 2^{64}$ için: 2^{64} adımlık iş 64 adıma iner.

Soru: Logaritmada nasıl bu kadar kısa oluyor? **Cevap:** Burada adım sayısını saydık. Düz algoritmada n yerine 1024 verirsem 1024 + 4, 2^{64} verirsem $2^{64} + 4$ işlem yapılır. Diğer algoritma $\log n$'e iner: $\log_2 2^{64} = 64 \log_2 2 = 64$. $\log n$'in nereden geldiği özyinelemeli algoritmalarda konuşulacak.

> **Ders sonrası not:** Örnekteki 4 sayısı tur sayısıdır, çarpma sayısı değildir. Python ile sayıldığında 1101_2 için toplam 7 çarpma yapılıyor: üç 1 biti için ikişer, bir 0 biti için bir çarpma. Yine de çarpma sayısı üssün bit sayısıyla büyür. Ayrıca hocanın burada n dediği büyüklük, büyük olasılıkla tabanın değil üssün değeridir.

5.5 Kodlama Üzerine Sorular

Soru: Bunu kodlarken yine `for` döngüsü mü kullanıyoruz? **Cevap:** `while` döngüsü de olur.

Soru: 1 ve 0'ı ayırt etmek için `if` kullanmak gerekir mi? **Cevap:** `if` de olur, `if`'siz de yapılabilir. Kare her durumda alındığına göre `n` çarpanının önüne bir yardımcı değişken koyulur: `t = n * digit`, sonra `n` yerine `t` yazılır. Sıfır gelirse çarpan eklenmez, bir gelirse eklenir. Bir öğrencinin daha önce sorduğu dallanmasız hesaplamaya (branchless computing) bu bir örnektir.

> **Ders sonrası not:** Hoca `t = n * digit` dedi; bu hâliyle bit 0 iken $t = 0$ olur ve `R = R * R * t` sonucu sıfırlar. Python denemesinde 5^{13} için sonuç 0 çıktı. Doğrusu, bit 0 iken çarpanın 1 olmasıdır: `t = 1 + (n - 1) * digit`. Bu biçimle aynı denemede 1220703125 bulundu.
>
> **Hocanın vurgusu:** Kafayı kullanınca işler güzelleşiyor, çok ciddi bir iyileştirme sağlanabiliyor. Önemli olan biraz korteksi çalıştırmak.

6. Akış Diyagramı, Sözde Kod ve Program Kodu

Hoca yeni probleme geçmeden akış diyagramlarını bundan sonra azaltacağını söyledi. Çoğu öğrencinin akış diyagramı çizmede sorunu kalmamalı, ama koda geçince giriş ve çıkışlarda hata çıkıyor. Gerçeklenmeyen algoritma boşta kalıyor. Dersin adı da zaten yapısal programlamaya giriş olduğu için algoritmalar daha çok kod üzerinden ifade edilecek.

> **Hocanın vurgusu:** Akış diyagramını da, sözde kodu da, C veya herhangi bir programlama dilini de bilmeniz gerekir. Bir yerde bir algoritma okuduğunuzda her zaman program kodu gelmez, oturup çalıştıramazsınız. Karşınıza akış diyagramı çıkarsa onu okuyup anlamanız lazım.

7. Pi Sayısını Seriyle Hesaplamak

Hoca tahtaya bir seri yazıp neye karşılık geldiğini sordu. Cevap pi sayısıydı:

$$\pi = 4 - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \cdots$$

Kabul edelim ki C dilinde pi sayısını çağıramıyoruz ve onu sayısal yöntemle bulmak istiyoruz. Önce serinin genel terimi kuruldu. Bu kısım sınıfla birlikte yürüdü:

- Bir öğrenci $(-1)^{n-1}\frac{4}{n+2}$ önerdi. Hoca $n = 1$ için bunun $\frac{4}{3}$ verdiğini, böylece baştaki 4'ün atıldığını gösterdi. Mantık güzel, ama formül oturmadı.
- Payda $2n - 1$ olarak düzeltildi. $n = 1$ için $+\frac{4}{1}$, $n = 2$ için $-\frac{4}{3}$, $n = 3$ için $+\frac{4}{5}$ çıkar.

$$\pi = \sum_{n=1}^{\infty} (-1)^{n-1} \frac{4}{2n-1}$$

Burada $(-1)^{n-1}$ hesabı sorunludur: $n = 5$ için 4 kez çarpma gerekir. Üstel durumdan kurtulmak gerekiyor. Sınıftan n'in 2'ye göre modunu alma önerisi geldi.

> **Hocanın vurgusu:** `for` döngüsünü, özellikle 1'den n'e giden `for` döngülerini sevmiyoruz. İç içe gelirlerse hiç sevmiyoruz.

Serinin sonsuz terimi açılamaz. Bu yüzden terim sayısını n olarak kullanıcıdan alacağız. Gerçek pi ile bulunan değer arasındaki fark kesme hatasıdır, çünkü seriyi bir yerde kesiyoruz. Hoca serilerle bulmanın yanında olasılıksal bir yol olan Monte Carlo yöntemiyle pi tahmininin de ileride anlatılacağını söyledi.

8. Pi Algoritmasını Adım Adım İyileştirmek

8.1 Birinci Sürüm: İki Kollu `if`

Payda $2i - 1$ diye gider, pay ise sabit 4'tür. Hoca 4'ü parantezine aldı, sonucu en sonda 4 ile çarptı:

1. Başla, n'i kullanıcıdan al.
2. `result = 0`.
3. `i` 1'den n'e kadar birer birer artarken:
   - `i mod 2 == 1` ise `result = result + 1 / (2i - 1)`.
   - Değilse `result = result - 1 / (2i - 1)`.
4. Ekrana `4 * result` yaz, dur.

8.2 İkinci Sürüm: Ortak İşi `if`'in Üstüne Almak

`if`'in iki kolunda da aynı $\frac{1}{2i-1}$ hesaplanıyor. Geçici bir `T` değişkeni tanımlanıp döngüye girer girmez `T = 1 / (2i - 1)` yapılırsa iki kolda da yalnızca `T` yazılır. Sonuç değişmez.

Hoca bir laboratuvar deneyimini anlattı: OBEB-OKEK'le ilgili bir soru kalıbında bu işe yeni başlayan herkes aynı hatayı yapıyordu, aynı işlemi `if`'in iki kolunda da yazıyordu. Aynı soru kalıbı ileride çözülecek.

> **Hocanın vurgusu:** `if` kullanacaksan yine kullan. Ama iki kolda yapılan işlem birebir aynı sonucu veriyorsa bunu bir üste almak çok daha fayda sağlar. "Günümüz bilgisayarlarında ne fark eder" demeyin, gömülü sistemlerle çalışan arkadaşlar bunları zamanında düşündüklerine seviniyorlar.

8.3 Üçüncü Sürüm: İşaret Değişkeni

Bir öğrenci işaret bir artı bir eksi gittiği için sürekli -1 ile çarpılabileceğini önerdi. `if` biraz sıkıntılı bir yapı olduğu için onu kaldıralım. Ek bir bellek hücresi gerekiyor: bir işaret (sign) değişkeni.

1. Başla, n'i al.
2. `result = 0`, `sign = 1`. İlk terim toplama olduğu için 1.
3. `i` 1'den n'e kadar artarken:
   - `result = result + sign * 1 / (2i - 1)`.
   - `sign = sign * -1`.
4. Ekrana `4 * result` yaz, dur.

C'de ikinci satır `sign = -sign` diye de yazılabilir. Bu, değişkenin negatifini alır.

8.4 Dördüncü Sürüm: İlk Terimi Döngüden Çıkarmak

Baştaki $+4$'ü zaten biliyoruz. O hâlde başlangıç değerlerini ona göre ayarlayalım: `R = 4`, `i = 2`, `s = -1`, formüldeki pay da 4 olsun. İlk turda `R = 4 - 4 / 3` bulunur. Döngü bir tur az çalışır. Başlangıç ataması zaten yapılacağı için 0'dan ya da 4'ten başlatmak aynı maliyettedir.

Soru: Bu durumda sondaki `4 * result` çarpımını sileceğiz değil mi? **Cevap:** Evet, yalnızca `R` yazdırılır.

8.5 Soru: Büyük Sayıları Toplamak Daha mı Pahalı?

Soru: $2 + 2$ ile $2 + 2002$ toplamanın işlem gücü açısından farkı var mı? Büyük sayıda rakam sayısı artmıyor mu? **Cevap:** Hocanın bildiği kadarıyla hazır işlemcilerde yok. Bir tam sayı (integer) bellekte 4 bayt, yani 32 bit yer kaplar. "İlk iki biti yerleştireyim, kalanlar zaten sıfır" diye bir şey yoktur. İki tam sayı 32 bitlik bir toplama devresine girer, bitlerinin çoğu 0 da olsa 32 bit giriş olur, 32 bit çıkış alınır. Donanımı kendiniz tasarlıyor ve sayının büyüklüğüne göre yer ayırıyorsanız fark edebilir. Ondalıklı sayı ile tam sayı toplamak ise başka bir iştir. Hoca donanım tarafını iyi bilmediğini, bu soruya akademik sayfalardan ve Stack Overflow'dan bakılabileceğini söyledi.

8.6 Soru: `i`'yi İkişer Artırıp İki İşlem Yapsak?

Bir öğrenci işaret değişkeni yerine şunu önerdi: `i` ikişer artsın (1, 3, 5, ...), döngünün içinde bir toplama bir çıkarma yapılsın.

Hoca öneriyi tahtada denedi, `R` burada 0'dan başlıyor:

- `i = 1`: $R = 0 + \frac{4}{2 \cdot 1 - 1} = 4$, sonra $R = 4 - \frac{4}{3}$.
- `i = 3`: $\frac{4}{2 \cdot 3 - 1} = \frac{4}{5}$ eklenir, sonra $\frac{4}{7}$ çıkarılır.

Örüntü doğru gidiyor. Çıkarılan terimin paydası $2i + 1$ olmalıdır.

Başka bir öğrenci bu yöntemin bir sorununu yakaladı: Terim sayısı tek ise son işlem fazladan yapılmış olabilir. Her turda iki terim eklendiği için ilk n terim istendiğinde bir terim fazla eklenebilir.

Bu sürüm ve önceki üç sürüm ders sonrasında C ile yazılıp `gcc -Wall` ile derlendi. Hocanın süre karşılaştırması için andığı $n = 10000$ değerinde dördü de 3.141493 verdi.

9. Pi Programının C ile Yazılması

Hoca dördüncü sürümü canlı olarak kodladı. Değişkenler: terim sayısı `n` (tam sayı), işaret `s` (tam sayı), sonuç `tp` (double), döngü değişkeni `i`.

İki ayrıntıya dikkat çekti:

- Formüldeki pay `4` değil `4.0` yazılmalıdır. `2 * i - 1` tam sayı olduğu için `4 / (2 * i - 1)` tam sayı bölmesi olup 0 elde edilebilir.

- Döngü şartı `i <= n` olmalıdır: Birinci terim başlangıçta alındı, ikinci terim `i = 2`'de hesaplanır, en son n. terim hesaplanır, $n+1$. terim hesaplanmaz.

Pi'yi `math.h` içindeki sabit `M_PI` verir. Bununla hatayı da yazdırıyoruz. Son hâl şöyledir:

```c
#include <stdio.h>
#include <math.h>

int main(void)
{
    int n;
    int s = -1;
    double tp = 4;
    int i;

    printf("n giriniz: ");
    scanf("%d", &n);
    for (i = 2; i <= n; i++) {
        tp = tp + s * 4.0 / (2 * i - 1);
        s = -s;
        printf("%d Sonuc = %lf Hata = %lf\n", i, tp, fabs(M_PI - tp));
    }
    return 0;
}
```

9.1 Çalıştırırken Görülen Hatalar

Kod ilk çalıştırmada yanlış sonuç verdi. Hoca $n = 10$ girince 4.95 ve -1.81 gördü. Bir öğrenci sebebin `tp`'nin 4 ile çarpılması olabileceğini söyledi, hoca ise çok daha büyük bir sapma gördüğünü belirtti. Başka bir öğrenci işaretin -1 olarak tanımlanması gerektiğini hatırlattı: `s` önce 1 verilmişti.

> **Ders sonrası not:** Python denemesi bu teşhisi destekliyor. `tp = 4`, `s = 1`, $n = 10$ ile sonuç 4.958160, $\pi - tp$ ise -1.816568 çıkıyor. Bu, ekranda okunan değerlerle uyuşuyor. Yani o andaki hatanın kaynağı işaretin $+1$ başlatılmasıdır.

Hatanın kendisi hakkında da bir ayrıntı konuşuldu. Hata pozitif çıksın diye farkın yönü tartışılırken hoca yöne değil miktara bakılması gerektiğini söyledi: Hatanın azaldığını görmek için mutlak değer (absolute value) alınır. Normal `abs` tam sayılar üzerinde çalıştığından double için `fabs` kullanılması gerekti. Mutlak değer alındığı için `M_PI` ile `tp`'nin yerinin değişmesi sorun olmaz.

Düzeltilmiş programı hoca $n = 50$ ile çalıştırdı, `fabs` eklendikten sonra değeri büyüttü. Ekranda 2.66'dan başlayan değerler, hata sütununda 0.47, 0.32, 0.246 ve 0.16 gibi giderek azalan sayılar göründü. Ders sonrasında aynı döngünün çıktısı:

i	tp	Hata
2	2.666667	0.474926
3	3.466667	0.325074
4	2.895238	0.246355
5	3.339683	0.198090
6	2.976046	0.165546
10	3.041840	0.099753
50	3.121595	0.019998
10000	3.141493	0.000100

Son üç satırdaki n değerleri derste girilen değerlerdir. Ekranda okunan rakamlar altyazıda bozuktur ("2.46"), tablo `gcc -Wall` ile derlenen yukarıdaki programdan alınmıştır.

9.2 Hesaplama Hızlı, Yazdırma Yavaş

Program çalışırken ekran uzun süre aktı. Hoca bunun hesaplamadan değil yazdırmadan kaynaklandığını gösterdi: $n = 10000$ dediğinde işlemci hesabı çoktan bitirir, uzun akış ekrana yazmak için beklemektir. Satırlara `i` de yazdırılınca 3.14 değerine çok daha erken ulaşıldığı görüldü. Hassasiyet ihtiyacı 3.14 ise o kadar tur gereksizdir. Hatayı daha ince görmek için yazdırma ayrıntısı artırılabilir.

Hocanın çıkardığı sonuç şudur: Algoritmada bazı şeyleri değiştirerek gereksiz işleri tekrar tekrar yapmanın önüne geçiyoruz ve bu zaman açısından çok iyi bir şey sağlıyor.

10. İki Sürüm Arasındaki Trade-off

Aradan sonra hoca `if`'li sürüm ile işaret değişkenli sürümü yan yana koyup aralarında nasıl bir ödünleşim (trade-off) olduğunu sordu.

> **Hocanın vurgusu:** Trade-off'un ne olduğunu artık herkesin bilmesi lazım. "Trade-off nedir?" diye sorulduğunda bir paragraf yazabilmelisiniz.

Bir öğrencinin cevabı doğru bulundu: Soldaki `if`'li sürüm daha çok işlem yapar, daha az bellek kullanır. Sağdaki işaret değişkenli sürüm daha az işlem yapar, daha çok bellek kullanır. Biraz zihnimizi zorlayınca `if`'ten kurtulduk ve işlemciyi yormadık, ama karşılığında `sign` diye bir değişken harcadık. `T` değişkeni de aynı mantıkla ortaya çıkmıştı: `if`'in içindeki işlem karmaşıklığını azaltmak için.

Hoca bunu genel bir ilkeye bağladı: Programlamada algoritma geliştirirken yapılacak işler mümkün oldukça temel operasyonlar cinsinden olmalıdır. `for` ve `if` temel operasyon değildir. Bu yüzden `if`'i kaldırmak süre açısından fayda sağlar.

Hocanın önerdiği deney şudur: İki sürümü de kodlayın, aynı terim sayısı için, örneğin $n = 10000$ için, ne kadar sürede hesapladıklarını C'deki basit bir zaman ölçme koduyla karşılaştırın.

11. e^x Serisi

Fatih Hoca'nın notlarındaki üçüncü soru şudur:

$$e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots + \frac{x^n}{n!}$$

x ve terim sayısı girilecek, sonuç bulunacak. Hoca, işin sayısal analiz tarafına bu derste girmediğini, seride Maclaurin ve Taylor'la bir bağ olduğunu söyledi ama adını kesinleştirmedi.

> **Ders sonrası not:** Hoca serinin adından emin olmadığını belirtti; bu seri büyük olasılıkla e^x'in Maclaurin serisidir, yani 0 noktası etrafındaki Taylor serisidir.

11.1 Birinci Sürüm: İç İçe Üç Döngü

"Çok kafa yormadan" kurulan ilk çözümde pay (x^i) ve payda ($i!$) her terim için sıfırdan hesaplanır. İlk terim olan 1 ($x^0/0!$) doğrudan `result`'a konur, geriye $n-1$ terim kalır.

1. Başla, x ve n'i al.
2. `result = 1`.
3. `i` 1'den $n-1$'e kadar artarken:
   - `pay = 1`. `j` 1'den `i`'ye kadar artarken `pay = pay * x`.
   - `payda = 1`. `j` 1'den `i`'ye kadar artarken `payda = payda * j`.
   - `result = result + pay / payda`.
4. `result`'ı yaz, dur.

Hoca önce pay döngüsünde `pay = pay * j` yazdı, faktöriyel kalıbı aklında kaldığı için. Kuvvet alınırken `x` ile çarpılması gerektiğini görüp düzeltti.

> **Ders sonrası not:** Algoritma n terim toplar, son terim $\frac{x^{n-1}}{(n-1)!}$ olur. Tahtadaki yazımda son terim $\frac{x^n}{n!}$ olduğu için 0'dan n'e kadar sayılırsa $n+1$ terim vardır. n burada terim sayısı olarak okunmalıdır.

Soru: Serinin $\frac{x^1}{1!}$'den başlaması gerekmiyor mu? Baştaki 1'i nasıl elde ettik? **Cevap:** İlk eleman 1 olduğu için `result` doğrudan 1 yapıldı. `i` 1'den başlayınca iç döngü önce 1 kez döner, sonra 2 kez, sonra 3 kez.

Hoca bir adım daha atlanabileceğini gösterdi: `result = 1 + x` diyebiliriz. O zaman `i` 2'den başlar ve n dahil olmadan n'e kadar gider, yani döngü $n-2$ kez döner. İç döngüler ise bu sefer 1'den 2'ye doğru çalışmaya başlar.

11.2 İkinci Sürüm: İki İç Döngüyü Birleştirmek

İki iç döngü de 1'den `i`'ye kadar dönüyor ve birbirini etkilemiyor. Öyleyse ayrı döngü yazmaya gerek yok: Tek iç döngünün gövdesinde hem `pay = pay * x` hem `payda = payda * j` yapılır.

Hoca burada kodun kısaldığını, ama yapılan işlem sayısının değişmediğini vurguladı. Bir iç döngü m kez dönerse gövdesindeki bir çarpma ve bir atama için $2m$ işlem yapar. İki ayrı döngü $2m + 2m = 4m$ işlem eder. Birleştirilmiş döngü m kez dönüp gövdede 4 işlem yapar, yine $4m$.

Soru: Burada daha kısa yapmış olmuyor muyuz? **Cevap:** İşlem bakımından hayır. Bu ikisi birbirini etkileyen iç içe döngüler değil, peş peşe gelen ayrı döngülerdir. Toplam yine $4m$'dir.

> **Ders sonrası not:** Hocanın bu sayımı yalnızca gövdedeki işlemleri kapsıyor. Bölüm 1'deki gibi döngü kontrolü de sayılırsa (karşılaştırma ve artırma) birleştirilmiş döngüde kontrol işleri bir kez yapıldığı için toplam işlem büyük olasılıkla biraz azalır. Büyüme mertebesi aynı kalır.

11.3 Üçüncü Sürüm: Önceki Turdaki Değeri Kullanmak

Hoca "burada nasıl iyileştirme yapılabilir?" diye sordu. Bir öğrenci işin tek döngüyle yapılabileceğini, pay ve paydanın adım adım güncellenip `result`'a eklenebileceğini söyledi.

Mantık şudur: x^2'yi hesapladım, onu neden 1'e çekip baştan başlıyorum? Üzerine bir x koysam x^3'ü bulurum. İki faktöriyeli bulduktan sonra bir kez daha çarpsam üç faktöriyeli bulurum. Çünkü:

$$x^n = x \cdot x^{n-1}, \qquad n! = n \cdot (n-1)!$$

1. Başla, x ve n'i al.
2. `result = 1`, `pay = 1`, `payda = 1`.
3. `i` 1'den $n-1$'e kadar artarken:
   - `pay = pay * x`.
   - `payda = payda * i`.
   - `result = result + pay / payda`.
4. `result`'ı yaz, dur.

Birinci turda `pay` x, `payda` $1!$ olur. İkinci turda `pay` x^2, `payda` $2!$ olur. Artık hiçbir şeyi sıfırlamaya gerek yoktur, değerler kümülatif olarak birikir. Aradaki toplama işlemi hazır olan pay ve paydayı kullanır. Üç sürüm ders sonrasında C ile yazılıp `gcc -Wall` ile derlendi, üçü de aynı sonucu verdi ve matematik kütüphanesindeki `exp` ile uyuştu.

Seride bir artı bir eksi işaret olsaydı aynı mantıkla bir işaret değişkeni eklenirdi.

11.4 Çalışma Soruları

Fatih Hoca'nın notlarında benzer yaklaşık hesap soruları da var: $\cos x$'in hesabı (radyan cinsinden, gerçek değerle farkı da bulunarak), diğer trigonometrik fonksiyonlar, $\sqrt{x}$ ve $x^{y/2}$ için algoritmalar. Hoca bunların hepsinin yaklaşık değerle ilgili olduğunu söyledi.

12. Fibonacci Sayıları

Fibonacci dizisi bilgisayar bilimlerinde çok kullanılır, çünkü üzerinde algoritmik olarak rahatça deney yapılabilir. Dizinin her terimi kendinden önceki iki terimin toplamıdır: 1, 1, 2, 3, 5, 8, … Bazı kaynaklarda dizi 0 ile başlar. Burada ilk iki terim 1 alınıyor.

12.1 Diziyle Çözüm

1. Başla, kaçıncı terim isteniyorsa n'i al.
2. `F[1] = 1`, `F[2] = 1`.
3. `i` 3'ten n'e kadar artarken `F[i] = F[i-1] + F[i-2]`.
4. `F[n]`'i yaz, dur.

12.2 Araya Giren Karşılaştırma: e^x Sürümlerinin Maliyeti

Fibonacci'nin maliyetine geçmeden hoca e^x çözümlerine döndü. Burada tekil işlemleri değil, yalnızca döngünün dönme sayısını sayıyoruz, çünkü içerideki işler hemen hemen hepsinde ortak.

Birinci sürümde dış döngü en kötü n kez döner. İç döngüler 1 kez, 2 kez, 3 kez … en fazla n kez döner. Yani yaklaşık $n \cdot n = n^2$ dönüş olur. Dış döngüdeki iki ayrı iç döngü sayılırsa $n \cdot 2n = 2n^2$ bulunur, o da yine n^2 gibi büyür.

> **Hocanın vurgusu:** 1'den n'e iç içe döngüler görürseniz orada kabaca n^2 tekrar yapılacağını söyleyebilirsiniz. Tam olarak değil, ama kabaca.

Soru: $n \cdot n!$ olabilir mi? **Cevap:** Olmaz. İç döngülerin dönme sayısı `i`'nin ulaşacağı en büyük değerle, yani n ile sınırlıdır. $n \cdot n$ olur. $n \cdot n!$ çok kötü bir durum olurdu.

Üçüncü sürümde ise tek döngü n kez döner. İşlem açısından n^2'den n'e düştük. Bellek açısından ek bir şey harcamadık: x, n, `result`, `pay`, `payda` zaten vardı. Hoca bunu "çok güzel bir iyileştirme" diye niteledi, ama her problemde böyle olmadığını ekledi.

12.3 Diziyle Çözümün Sorunu

Fibonacci'nin dizili çözümünde döngü n kez döner, bu iyi görünür. Hoca çözümde bir sorun olduğunu söyleyip sordu.

Soru: n için 1 veya 2 veremiyor muyuz? **Cevap:** Verebiliriz. O zaman döngüye girmez, doğrudan sonuca gider.

Asıl sorun bellektedir. n yerine 1 milyon ya da 10 milyon girilirse döngünün o kadar dönmesi çok bir şey değildir. Ama o kadar elemanlı bir dizi gerekir. Yani bu algoritma bellek açısından verimli (memory efficient) değildir. Belleğiniz yetersizse kötüdür.

Bir öğrenci "bir değişkenle toplayarak gitsek?" diye önerdi. Doğrudur: Bir geçici değişkenle yine n kez döneriz, ama bellek açısından çok daha iyi oluruz. Diziye gerek yoktur, çünkü yalnızca son iki terim lazımdır. Hoca, birinci sınıftayken bu soruyu dizi ve özyineleme kullanmadan çözmeye çalıştığını, kâğıt üzerinde değişkenleri birbirine aktarmakta zorlandığını anlattı.

12.4 Üç Değişkenli Çözüm

Mantık şudur: Son iki terime `c1` ve `c2`, yeni terime `c3` diyelim. Yeni `c3`'ü hesapladıktan sonra değerler bir kaydırılır. Önce `c2`'yi `c1`'e atamak gerekir. Sırayı değiştirip `c3`'ü önce `c2`'ye atarsak `c2`'nin mevcut değerini kaybederiz.

1. Başla, n'i al.
2. `c1 = 1`, `c2 = 1`.
3. `i` 3'ten n'e kadar artarken:
   - `c3 = c1 + c2`.
   - `c1 = c2`.
   - `c2 = c3`.
4. `c2`'yi yaz, dur.

Hoca bunun bir değiş tokuş (swap) değil kaydırma (shift) işlemi olduğunu belirtti.

12.5 İki Değişkenli Çözüm

Bir öğrenci üçüncü değişkene de gerek olmadığını gösterdi. Döngü gövdesi şöyle olur:

- `c2 = c1 + c2`.
- `c1 = c2 - c1`.

İkinci satırda `c2`'nin içinde $c_1 + c_2$ vardır. Buradan eski `c1` çıkarılınca eski `c2` kalır: $(c_1 + c_2) - c_1 = c_2$. Böylece `c1` eski `c2`'nin değerini, `c2` ise yeni terimin değerini alır.

Hoca işlemleri saydı: Üç değişkenli çözümde bir toplama ve üç atama, yani 4 işlem vardır. İki değişkenli çözümde bir toplama, bir çıkarma ve iki atama, yine 4 işlem vardır. İşlem sayısı aynıdır, ama bir satır daha azdır ve bir değişken daha az kullanılır. İki çözümle de 1, 1, 2, 3, 5, 8 dizisi elde edildiği python ve `gcc -Wall` denemesiyle doğrulandı.

13. Basamaklarının Küpleri Toplamı Kendine Eşit Olan Sayılar

Soru şudur: 1'den 1000'e kadar, basamaklarının küpleri toplamı kendine eşit olan sayıları bulun. Hoca bu sayılara "Armstrong" dendiğini hatırladığını söyledi. Örnek olarak 1, 153, 370 sayılarını gösterdi.

Sayılar 999'a kadar gidiyor, yani en çok üç basamaklıdır. n basamaklı olsaydı bir `while` döngüsü gerekirdi, ona daha sonra bakılacak. Sayının basamaklarına a, b, c diyelim. Hocanın örneği 989:

- c en kolayı: `c = i mod 10`. 989 için 9.
- a: Sayıyı 100'e böl, aşağı yuvarla. $\lfloor 989/100 \rfloor = 9$. Hoca önce "ona böl" dedi, bir öğrencinin uyarısıyla 100 diye düzeltti.
- b: Sayıdan $a \cdot 100$'ü çıkar, kalanı 10'a böl, aşağı yuvarla. $989 - 900 = 89$, $\lfloor 89/10 \rfloor = 8$.

Aşağı yuvarlama için iki gösterim vardır. $\lfloor x \rfloor$ taban (floor), yani aşağı yuvarlamadır. $\lceil x \rceil$ tavan (ceil), yani yukarı yuvarlamadır. Bölme işlemlerinde aşağı yuvarlama lazımdır. C'de iki tam sayı bölünürse tam kısım alınır, ondalık kısım silinir.

Algoritmanın adımları:

1. Başla. Burada kullanıcıdan alınan bir n yoktur.
2. `i` 1'den 999'a kadar birer birer artarken:
   - `c = i mod 10`.
   - $a = \lfloor i/100 \rfloor$.
   - $b = \lfloor (i - a \cdot 100)/10 \rfloor$.
   - `a*a*a + b*b*b + c*c*c == i` ise `i`'yi yaz. Değilse bir şey yapma.
3. Dur.

Küp, son satırdaki gibi uzun uzun çarpma olarak yazılabilir.

Bir öğrenci b'nin hesabını kısalttı: $b = \lfloor (i \bmod 100)/10 \rfloor$. 989 için $989 \bmod 100 = 89$, $\lfloor 89/10 \rfloor = 8$. Hoca bunun doğru olduğunu söyledi.

Ders sonrasında aynı algoritma çalıştırıldı, sonuç: 1, 153, 370, 371, 407.

> **Ders sonrası not:** Hoca "Armstrong" adını soru işaretiyle andı. Armstrong sayısı tanımı büyük olasılıkla basamak sayısı kadar kuvvet kullanır. Bu soruda ise bütün basamaklar için küp kullanılıyor; üç basamaklı sayılarda iki tanım çakışır.

14. Karşıt Örnekle Çürütme

Hoca, matematiksel işlemlerde kullandığımız düşünce biçiminin mantıkta da işe yaradığını söyleyerek yeni bir önerme yazdı:

> Her pozitif doğal sayı n için $n^2 + n + 41$ asaldır.

Elle deneyelim:

- $n = 1$: $1 + 1 + 41 = 43$, asal.
- $n = 2$: $4 + 2 + 41 = 47$, asal.
- $n = 3$: $9 + 3 + 41 = 53$, asal.

Bir yere kadar gidebiliriz, ama bütün pozitif doğal sayılar için doğruluğu böyle gösteremeyiz. Çünkü n için bir sınır verilmemiştir.

Hoca burada mantıktaki yaklaşımı anlattı: Bir şeyin yokluğu ispat edilmez, ancak var olduğunu gösteren bir delil ortaya konur. Sonsuza kadar bütün durumlara bakmak mümkün değildir. Onun yerine bunu önermeyi yanlış çıkaracak bir arama algoritması gibi düşünün. 1'den başlayıp gidiyoruz, bir yerde önerme bozulunca arama duracak. Buna aksine örnek, yani karşıt örnek (counterexample) denir. Konu gelecek dönem ayrık matematikte daha ayrıntılı işlenecek.

Sınıftan bir öğrenci $n = 40$'ta bozulduğunu söyledi. Hoca kontrol etti:

$$40^2 + 40 + 41 = 40 \cdot (40 + 1) + 41 = 40 \cdot 41 + 41 = 41 \cdot (40 + 1) = 41 \cdot 41 = 1681$$

$1681 = 41^2$ asal değildir. Önerme çürüdü. Python ile tarandığında da önermeyi bozan ilk değerin $n = 40$ olduğu doğrulandı.

> **Ders sonrası not:** Hoca "bir şeyin yokluğu ispat edilmez" ve "sonsuz arama uzayında bir önermenin doğruluğunu bulmak mümkün değildir" dedi; bu, tek tek deneyerek gösterme için geçerlidir. Sonsuz durumu kapsayan önermeler büyük olasılıkla matematiksel ispatla gösterilebilir. Dersin başındaki Gauss formülü de her n için geçerlidir.

14.1 Neden `for` Değil `while`?

Hoca önce "`for` ile başlayacağım" deyip bunun doğru olup olmadığını sordu. Bir öğrenci `while`'ın daha uygun olduğunu, çünkü karşıt örnek bulununca döngüden hemen çıkılacağını söyledi. Ne zaman biteceğini bilmediğimiz için `while` kullanılır.

1. `n = 1`, `flag = 1`.
2. `flag` 1 olduğu sürece:
   - $n^2 + n + 41$'in asal olup olmadığını test et.
   - Asal değilse `flag`'i 0 yap. Karşıt örnek bulundu.
   - Asalsa `n`'i bir artır.
3. Bulunan `n`'i yaz.

Asallık testi de aslında karşıt örnekle yapılır: Sayının yarısından ya da karekökünden başlayıp 2'ye doğru gidersiniz. Yolda herhangi birine bölünüyorsa çıkarsınız. Bu, asal olmadığını gösteren bir örnek bulmaktır.

15. $a^4 + b^4 + c^4 = d^4$ Önermesi

Bir sonraki önerme şudur:

$$\nexists\, a, b, c, d \in \mathbb{N}^+ : a^4 + b^4 + c^4 = d^4$$

Yani hiçbir pozitif doğal sayı dörtlüsü bu eşitliği sağlamaz. Hoca $\exists$ sembolünün "bazı", önceki önermedeki $\forall$ sembolünün "her" demek olduğunu hatırlattı.

Bu önermenin yanlış olduğunu göstermek için eşitliği sağlayan tek bir dörtlü bulmak yeter. Hoca önce bunun çelişkiyle ispat edilebileceğini söyledi, sonra düzeltti: Bu çelişki (contradiction) değil, karşıt örnekle ispattır. Çelişkiyle ispat ayrıca anlatılacak.

Soru: Nasıl bir döngü kullanacağız? **Cevap:** Bulunca çıkılacağı için iç içe `while` uygun. Bir sayı dizisi verilip bir sayının dizide olup olmadığını sormak da aynı türden bir varlık problemidir (existence problem). Ayrık matematikte bu adla geçer. Bulunduğunda aramaya devam edilmez.

Üzerinde çalıştığımız değişken dört tane olduğu için arama uzayı dört boyutludur. Yapı şöyledir:

1. `found = 0`.
2. `a` için `while`, içinde `b` için `while`, içinde `c` için `while`, içinde `d` için `while`.
3. En içte eşitliği test et. Sağlanıyorsa `found`'u 1 yap.
4. Dört döngünün şartına da `found`'u ekle. Böylece bulunan anda `d`, `c`, `b`, `a` döngüleri sırayla biter.

   Ders sonrası not: a, b, c sabitken `d` döngüsüne bir üst sınır konmazsa, dörtlü bulunmadığı sürece en içteki döngü hiç bitmez ve dış değişkenler hiç artmaz. Çalışan bir programda büyük olasılıkla sınır gerekir, örneğin `d`'nin d^4 değeri $a^4 + b^4 + c^4$'ü aştığında durması.

Hocanın anlattığına göre 1988'e kadar böyle bir dörtlü bulunamamıştır. Bilgisayarların işlem kapasitesi yetmiyordu. Hoca slaytta bulunan sayıların "çok da büyük olmadığını" söyledi. Bilgisayarın hızından yararlanılarak önermeyi yanlış çıkaran değerler elde edildi.

Ek bilgi (derste anlatılmadı): Bu önerme büyük olasılıkla Euler'in bir varsayımıdır. Bilinen karşıt örnekler büyük olasılıkla 1988'de Noam Elkies ve Roger Frye'a aittir: $2682440^4 + 15365639^4 + 18796760^4 = 20615673^4$ ve $95800^4 + 217519^4 + 414560^4 = 422481^4$. İki eşitlik de python ile doğrulandı. Sayıların bu büyüklüğü, dört iç içe döngüyle düz taramanın büyük olasılıkla pratik olmadığını gösterir.

16. Her Çift Sayı İki Asalın Toplamı mıdır?

Son önerme şudur: 2 haricindeki her çift sayı iki asal sayının toplamıdır. Hocanın örnekleri:

- $4 = 2 + 2$
- $6 = 3 + 3$
- $8 = 5 + 3$
- $18 = 13 + 5 = 11 + 7$

Bu ayrışımlar python ile doğrulandı. 18 için başka asal çift yoktur.

Bu önerme de sonsuza gittiği için bütün sayıları deneyerek doğruluğu test edilemez. Yine karşıt örnek aranacak ve yine bir `while` döngüsü olacak. Hoca sınıfa nasıl bir yaklaşım sunacaklarını sordu. Bir öğrencinin önerisi tahtada şu algoritmaya dönüştü:

1. `i = 4`, `flag = 1`.
2. `flag` 1 olduğu sürece:
   - `i`'ye kadar olan asal sayıları bul, bir diziye koy. Bunun içinde iki döngü vardır: 2'den `i`'ye kadar her sayı için bir döngü, her sayının asallığını test etmek için bir döngü daha.
   - Dizideki her `k`. asal için `i - A[k]` kalanını hesapla. Bu kalanın dizide olup olmadığını ara.
   - Hiçbir asal için bulunamazsa `flag`'i 0 yap. Karşıt örnek bulundu.
   - Bulunursa `i`'yi 2 artır.

Ders sonrası not: Hoca kalanın "diğer kalan elemanlarda" aranacağını söyledi; arama elemanın kendisini de kapsamalıdır. Yoksa derste örneği verilen $4 = 2 + 2$ ve $6 = 3 + 3$ yakalanamaz. `gcc -Wall` ile denendiğinde yalnızca diğer elemanlara bakan sürüm 4'ü yanlışlıkla karşıt örnek diye verdi, kendisini de kapsayan sürüm ise 100'e kadar karşıt örnek bulmadı.

Hoca bu problemin hâlâ çözülemediğini söyledi. Yanlışlığı gösterilemiyor, çünkü karşıt örnek bulunamadı. Doğruluğu da gösterilemiyor, çünkü bütün durumları değerlendirmek için sonsuza gitmek gerekir. Sadece 4 için bile bu kadar iş yaptık. Çok büyük bir çift sayıda ona kadarki asalları bulmak başlı başına büyük bir problemdir.

Ders sonrası not: Hoca önermenin çözülemeyişini asal bulma problemine bağladı; asal bulmanın zorluğu aramayı büyük olasılıkla yavaşlatır, ama önermenin açık kalmasının tek sebebi değildir. Asıl eksik büyük olasılıkla her çift sayıyı kapsayan bir matematiksel ispattır.

Ek bilgi (derste anlatılmadı): Bu önerme büyük olasılıkla Goldbach varsayımı adıyla bilinir.

Hoca bu örneklerin döngüler üzerine çalışmak için iyi olduğunu söyledi.

17. Bir C Programının Derlenme Aşamaları

Hoca bu kısmı IDE kullanmıyormuş gibi anlattı. Amaç, IDE'lerde bir düğmeyle yapılan işin arkasında neler olduğunu görmekti.

17.1 Örnek Program

Notepad++ ile bir metin dosyası açtı, C olarak ayarlayınca kod renklendi. Programda şunlar vardı:

- `stdio.h` için bir `#include` satırı.
- Bir makro tanımı (`MYSUM`, karşılığı `a + b`). Hoca makroların ayrıntısını bilmenin şart olmadığını, özelliklerinin şu olduğunu söyledi: Yazdığınız şey derlenme sırasında kullanıldığı yere kopyalanır.
- Bir yorum satırı.
- İki değişkeni toplayıp sonucu ekrana yazan bir `main`.

Dosya `deneme.c` diye kaydedildi. Toplam 9 satırdı ve çalıştırıldığında `result` değeri olarak 7 yazdı.

Bir programı işlemcide bir proses olarak koşturabilmek için belli aşamalardan geçirmek gerekir. Bunlara kabaca derleme diyoruz, ama derleme "pat" diye olan bir şey değildir. Bize bir derleyici lazımdır. Derleyici, bizim anlayıp bilgisayarın anlamadığı şeyi bilgisayarın anlayıp bizim anlamadığımız dile dönüştüren yardımcı bir programdır. C için birçok derleyici vardır, `gcc` bunlardan biridir. `gcc` komut satırından çalışır, grafik arayüzü yoktur.

17.2 Komut Satırında Gezinmek

Çift tıklama olmadığı için klasör içeriğini yazıp görmek gerekir:

- `dir` klasördeki dosyaları listeler. Linux'taki karşılığı `ls`'tir.
- `cd` (change directory) klasör değiştirir. Hoca `cd Desktop` ile başlayıp `compile test` klasörüne geçti.

Her seferinde bu yolu yürümek uzundur. Windows'ta klasörün içinde Shift'e basılı tutup sağ tıklayınca "Open command window here" çıkar, komut satırı o klasörde açılır. Shift'e basmadan bu seçenek çıkmaz.

17.3 PATH: `gcc` Neden Bulunamadı?

`gcc deneme.c -o mydeneme` yazınca "gcc is not recognized" hatası alındı. Sebep şudur: Komut satırı `gcc` yazıldığında önce bulunduğumuz klasörde `gcc.exe` arar. Orada yoksa işletim sisteminin bildiği belli yerlere bakar. Bu yerler ortam değişkenlerinde (environment variables) tutulur. Hoca bunu "`gcc`'yi arayacak bir havuz" diye tarif etti. Hocanın bilgisayarında `gcc` Dev-C++ kurulumunun MinGW klasörünün altındaki `bin` klasöründeydi. `gcc.exe`'yi çalıştığımız yere kopyalamak çözüm değildir, doğrusu yolu işletim sistemine tanıtmaktır.

Windows'ta yol eklemek için:

1. `gcc.exe`'nin yolunu dosya özelliklerinden kopyala.
2. Bilgisayarım, sağ tık, Özellikler, ileri düzey sistem ayarları, Environment Variables yolunu izle.
3. `Path` değişkenini düzenle. Yollar arasında noktalı virgül vardır. Son noktalı virgülden sonra yolu yapıştır, sonuna ters bölü ekle.
4. Tamam diyerek kapat. Bilgisayarı yeniden başlatmaya gerek yoktur, açık komut penceresini kapatıp yenisini açmak yeter.

Hoca Linux tarafında bunun daha kolay olduğunu söyledi. Derste ilk denemede hata devam etti. Doğrudan `bin` klasörüne gidip `gcc` yazınca program çalıştı, dosya verilmediği için uyarı verdi. Yani program sağlamdı, sorun yolda idi. Sebep, yolların arasındaki noktalı virgülden önce yanlışlıkla kalan bir boşluktu. Silinince `gcc` bulundu.

Aynı işlem kendi programımız için de yapılabilir: Derlenen programın klasörü `Path`'e eklenince `mydeneme` başka bir klasörden de çalıştı. Hoca bunun bu program için çok büyük bir fedakârlık olduğunu, sonra kaldıracağını esprili bir şekilde belirtti.

17.4 `gcc` Komutları

- `gcc deneme.c`: Programı derler. Çıktı adı verilmediği için derste `a` adlı bir çalıştırılabilir dosya oluştu. Çalıştırılınca 7 yazdı.
- `gcc deneme.c -o mydeneme`: Çalıştırılabilir dosyanın adı `mydeneme` olur.
- `-Wall`: Hoca bunu derleyiciyle ilgili bir parametre olarak andı.
- `-save-temps`: Derleme sırasında oluşan ara dosyaları saklar ve bize gösterir.

Kaynak dosya 1 KB iken çalıştırılabilir dosya 125 KB çıktı. Neden bu kadar büyüdü? Cevap, aşamalardadır.

> **Ek bilgi (derste anlatılmadı):** `-Wall`, büyük olasılıkla derleyicinin yaygın uyarılarını açan parametredir. Hata olmasa bile şüpheli kodu göstermesi yeni başlayanlar için büyük olasılıkla yararlıdır.

17.5 Dört Aşama

`-save-temps` ile derleyince klasörde `.i`, `.s` ve `.o` uzantılı dosyalar görünür. Her biri bir aşamanın çıktısıdır.

> **Ders sonrası not:** Hocanın dosyaları `deneme.i`, `deneme.s`, `deneme.o` adlarıyla oluştu. Linux'ta bir `gcc` denemesinde `-o mydeneme` ile birlikte kullanıldığında adlar `mydeneme-deneme.i` biçiminde çıktı. Ad farkı olursa şaşırmayın.

1. Ön işleme (preprocessing), `.c` → `.i`. Yorum satırları ve makro tanımları bilgisayar tarafından anlaşılmaz. Bunlar programlamayı kolaylaştıran dil özellikleridir, derleyici de bilmez. `MYSUM` diye bir komut yoktur, yalnızca yerine koyma (substitution) vardır. `#include` için de aynısı geçerlidir: Derleyici "bilmem, git onu getir buraya yaz" der.

`deneme.i` açıldığında 9 satırlık programın 600-700 satıra çıktığı görüldü. Dosyanın büyük kısmı `include` ile eklenen içerikti. En sonda bizim kodumuz vardı: `MYSUM` yok olmuş, yerine `a + b` gelmiş, yorumlar da silinmişti.

2. Derleme (compilation), `.i` → `.s`. `.i` dosyası girdi olur, derleyici dönüştürebildiği kadarını assembly koduna dönüştürür. Assembly, makine kodunun bir üst seviyesidir. `.s` dosyasında `mov` ve `add` gibi komutlar görülür. Bizim üç dört satırlık kodumuz burada satırlarca işleme karşılık gelir.

Fakat `printf` dönüştürülmemiştir. `.s` dosyasında yalnızca bir `call printf` vardır, `printf`'in ne olduğu burada belli değildir. Çünkü `printf` zaten önceden derlenmiştir, sonra bağlanacaktır. Bu çıktı işlemciye göre değişir: Aynı kodu kendi bilgisayarınızda denerseniz farklı assembly görebilirsiniz. Hoca bunu denemeyi önerdi. Bu davranış ders sonrasında `gcc -save-temps` ile denendi, `.s` dosyasında `call printf` satırı görüldü.

3. Çeviri (translation), `.s` → `.o`. Her assembly komutu birebir makine koduna eşlenir. Hoca bunu "bir sözlükten bakmak" gibi tarif etti. `.o` dosyası normal bir metin düzenleyicide anlamsız görünür. Uygun düzenleyicilerle 0 ve 1'ler yerine on altılık sayılar görülür. `.o` dosyasında `printf`'in kodu hâlâ yoktur. Bu yüzden boyutu kaynak kod gibi yaklaşık 1 KB'tır.

> **Ders sonrası not:** Hoca bu aşamaya "translation" dedi; bu aşama büyük olasılıkla genelde assembly (birleştirme) diye anılır, bu işi yapan programa da assembler denir. Aşamaların sırası aynıdır.

4. Bağlama (linking), `.o` → çalıştırılabilir dosya. Bizim `.o` dosyamız, varsa diğer obje dosyaları ve kullandığımız kütüphanelerin derlenmiş kodları birleştirilir. Derste yalnızca `printf` kullanıldı, başka kütüphane kullanılsaydı onlar da gelirdi. 125 KB'ın sebebi budur. Çalıştırılabilir dosyanın içinde `.o` ile ortak yerler ve kütüphanelerden gelen makine kodları vardır.

Çalıştırılabilir dosya çift tıklanınca ya da komut satırından çağrılınca proses hâline gelir, işletim sistemi tarafından yönetilir, kodları satır satır işletilir.

Hoca bu aşamaları ve çevre değişkenlerini bilmek gerektiğini, Linux gibi başka işletim sistemlerinde de denenebileceğini söyledi. Ama günlük işte IDE'ler kullanılır, çünkü asıl işimiz algoritmadır.

18. `scanf` ve Giriş Tamponu

Son konu, bir öğrencinin ekranını paylaştığı koddan çıktı. Hocaya göre bu, herkesin karşılaştığı önemli bir sorundu. Kod kaç öğrenci olacağını alıyor, sonra bir döngüde öğrenci adlarını okuyordu.

```c
#include <stdio.h>

int main(void)
{
    int n, i;
    char students[10][50];

    printf("Ogrenci sayisi: ");
    scanf("%d", &n);
    for (i = 0; i < n && i < 10; i++) {
        printf("Enter students name: ");
        scanf(" %[^\n]", students[i]);
        printf("%s\n", students[i]);
    }
    return 0;
}
```

Bu kod `gcc -Wall` ile uyarısız derlendi ve iki kelimelik adları tam okudu.

18.1 Sorun: Enter Tamponda Kalıyor

Biçimin başındaki boşluk olmadan `scanf("%[^\n]", students[i])` yazıldığında şu görüldü: Sayı olarak 5 girip Enter'a basınca program hiçbir ad girmeye izin vermeden bütün satırları ekrana yazdı.

Sebep şudur: 5 girip Enter'a bastığımızda Enter da bir karakterdir, yani `\n`'dir. `scanf("%d", &n)` sayıyı alır, `\n` ise bellekte, tamponda (buffer) kalır. Sıradaki `scanf` Enter görmeyene kadar okumak ister, ama ilk gördüğü şey zaten Enter'dır.

Bu arada başka bir kullanım hatası da düzeltildi: `scanf`'te değişkenin adresi verilmelidir, `printf`'te ise değişkenin kendisi. Hoca genel olarak bütün değişkenlerde `&` kullanılmasını, yalnızca string'in istisna olduğunu hatırlattı.

> **Ders sonrası not:** Hoca string için de "onda da kullanın, sonra düzeltirsiniz" dedi; `char students[10][50]` gibi bir tanımda `students[i]` büyük olasılıkla zaten adres gibi davrandığından `&` gerekmez. `gcc -Wall` denemesinde `&students[i]` biçimi uyarı verdi.

18.2 İki Çözüm

Birinci çözüm, tamponu boşaltmak: `scanf`'ten önce `fflush(stdin);` yazılır. Bu, giriş akışındaki (input stream) bütün tamponu boşalt demektir. Hoca bazı derleyicilerin bunu görmeyebileceğini de söyledi.

İkinci çözüm, biçimin başına boşluk koymak: `scanf(" %[^\n]", students[i])`. Hoca bunu "`scanf`'i bir şekilde aldatmak" diye anlattı: Önce bir boşluk oku, sonra Enter görene kadarki karakterleri oku.

Canlı denemede ikinci çözümle ilk ad okunurken beklenmedik bir davranış görüldü. Hoca bunu `[^\n]` kalıbına bağladı ve `fflush(stdin)`'i yeniden etkinleştirdi. Sorun bu şekilde kapandı.

> **Hocanın vurgusu:** Bu sorunun akış diyagramıyla hiçbir ilgisi yoktur, tamamen C'nin bir özelliğidir. Girdiğiniz her Enter tamponda kalır.

18.3 Karakter Okurken Aynı Sorun

Hoca aynı sorunu `char` ile gösterdi. Sayı okuduktan hemen sonra `printf` ile karakter istenip `scanf("%c", &c)` yazıldı. Program kullanıcıya hiç fırsat vermeden yoluna devam etti. `printf` çıkış akışına (`stdout`) yazar, giriş akışında (`stdin`) ise önceden girilmiş sayının Enter'ı durur. `%c` onu alır.

Çözüm yüzde işaretinden önce bir boşluk bırakmaktır: `scanf(" %c", &c)`. Yani bir boşluk oku, sonra bir karakter oku. Derste bu çalıştı. Aynısı ders sonrasında denendi: Boşluksuz sürümde `c`'ye 10 kodlu satır sonu karakteri geldi, boşluklu sürümde girilen karakter okundu. Hoca `stdin` ve `stdout` konularının ikinci sınıfta görüleceğini söyledi. Aynı durum `%d`'den sonra `gets` kullanırken de sıkıntı çıkarır.

18.4 `%s` ve `%[^\n]` Farkı

Soru: Köşeli parantezli kısmı yazmasak, doğrudan `%s` yazsak olur mu? **Cevap:** Olmaz. `%[^\n]` demek `\n` dışındaki karakterleri al, yani Enter görene kadar girilenleri al demektir. Şapka işareti hariç tutar. `%s` ise boşluk görene kadar okur.

Hoca bu farkı öğrencinin koduyla gösterdi: Şapkalı kalıp silinip `%s` yazıldı ve iki kelimelik bir ad girildi. İlk kelime okundu, ikincisi tamponda kaldı. Sonra aynı `scanf` ve `printf` satırları kopyalanıp ikinci okuma `students[i + 1]`'e yapıldı. İkinci `scanf` hiçbir şey girilmeden ikinci kelimeyi tampondan aldı. Yani tek bir ad, iki öğrenciye bölündü. Ad soyad birlikte okunacaksa `%[^\n]` kullanılmalıdır.

Bu davranış ders sonrasında `gcc -Wall` ile derlenen kısa bir programla da görüldü.

Soru: `scanf`'te her zaman `" %s"` yazmamız gerekiyor mu? **Cevap:** `%c`'de çalışıyor, ama string'lerde her zaman işlemiyor. Arada girilen bir boşluk karakteri işi bozabiliyor. Hocaya göre en doğrusu `fflush` ile akışı boşaltmaktır.

Hoca `%[^\n]` kalıbına "regular expression" dedi.

> **Ders sonrası not:** Hoca `fflush(stdin)`'i tamponu boşaltmanın en temiz yolu olarak önerdi; C standardında `fflush` büyük olasılıkla yalnızca çıkış akışları için tanımlıdır ve giriş akışında davranış tanımsızdır. Linux'ta denendiğinde `scanf("%d", &n); fflush(stdin); scanf("%[^\n]", s);` sırası adı okuyamadı, boş kaldı. Derste kullanılan Windows derleyicisinde çalışması büyük olasılıkla o ortama özgüdür. Aynı denemede `scanf(" %[^\n]", s)` iki kelimelik adı doğru okudu. Bu yüzden taşınabilir seçenek biçimin başındaki boşluktur.
>
> **Ders sonrası not:** Hoca `%[^\n]` için "regular expression" dedi; bu kalıp büyük olasılıkla `scanf`'e özgü "scanset" biçimidir, tam bir düzenli ifade değildir. Biçimdeki boşluk da büyük olasılıkla tek bir boşluk değil, sıradaki bütün boşluk ve satır sonu karakterlerini atlar.

Dersin Toparlaması

Bu derste aynı düşünce farklı problemlere uygulandı: Önce çalışan basit çözümü kur, işlem ve bellek maliyetini say, sonra gereksiz tekrarı bul. $4n + 2$ işlemlik toplam Gauss'la 4 işleme indi. Faktöriyelde Stirling doğruluktan ödün vererek bir yaklaşım sundu, hızlı üs alma n adımı $\log n$'e indirdi. Pi serisinde önce ortak iş `if`'in üstüne alındı, sonra `if` bir işaret değişkeniyle kaldırıldı. e^x'te önceki turun değerini kullanmak n^2'yi n'e indirdi. Fibonacci'de bellek sorunu, son iki terimi tutarak çözüldü. Bu örneklerin çoğunda bir ödünleşim vardı: işlem, bellek ve doğruluk birbirine karşı tartıldı. Karşıt örnek konusunda ise sonsuz bir arama uzayında tek bir örneğin önermeyi çürütmeye yettiği, ne zaman biteceği bilinmeyen aramalarda `while` ve bayrak kullanıldığı görüldü. Son kısımda derleme aşamaları (`.c`, `.i`, `.s`, `.o`, çalıştırılabilir dosya) ve `scanf`'in tampon davranışı incelendi.

Hocanın kapanış tavsiyesi çalışma biçimiyle ilgiliydi. Çizilen akış diyagramlarını koda dönüştürmek, kod üzerinde "şunu değiştireyim, bunu ekleyeyim" diye denemeler yapmak en iyi yoldur. Hiç kod yazmayan hiç hata yapmaz, hatasını görmeyen de düzeltemez, bu yüzden öğrenme gerçekleşmez. Bu iş kâğıt üzerinde olmaz, yapa yapa öğrenilir. Hocanın benzetmesiyle bu, İngilizce cümle kurmaya benzer: Yalnızca grameri öğrenmekle olmaz, bütün becerilerle

desteklenmesi gerekir. Aynı soruyu kod yazmış biri yarım saatte çözerken hiç kod yazmamış biri 15 saatte zor çözebilir. Akış diyagramları azalacak, derslerde giderek daha fazla C kodu yazılacak. Karşıt örnek konusunun ayrıntısı ayrık matematikte, $\log n$'in kanıtı özyinelemeli algoritmalarda, `stdin` ve `stdout` ayrıntıları ise ikinci sınıfta görülecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.